



## 저작자표시-비영리-변경금지 2.0 대한민국

이용자는 아래의 조건을 따르는 경우에 한하여 자유롭게

- 이 저작물을 복제, 배포, 전송, 전시, 공연 및 방송할 수 있습니다.

다음과 같은 조건을 따라야 합니다:



저작자표시. 귀하는 원저작자를 표시하여야 합니다.



비영리. 귀하는 이 저작물을 영리 목적으로 이용할 수 없습니다.



변경금지. 귀하는 이 저작물을 개작, 변형 또는 가공할 수 없습니다.

- 귀하는, 이 저작물의 재이용이나 배포의 경우, 이 저작물에 적용된 이용허락조건을 명확하게 나타내어야 합니다.
- 저작권자로부터 별도의 허가를 받으면 이러한 조건들은 적용되지 않습니다.

저작권법에 따른 이용자의 권리는 위의 내용에 의하여 영향을 받지 않습니다.

이것은 [이용허락규약\(Legal Code\)](#)을 이해하기 쉽게 요약한 것입니다.

[Disclaimer](#)

Thesis for the Degree  
of Master

**ProSmart: A Precise Mechanism for  
Scanning Propagated Vulnerable  
Smart Contracts at Large Scale**

by

**Yejin Joo**

**Department of Computer Science  
Engineering**

**Graduate School**

**Korea University**

**February 2022**

李 喜 造 教授指導

碩 士 學 位 論 文

# ProSmart: A Precise Mechanism for Scanning Propagated Vulnerable Smart Contracts at Large Scale

이 論文을 컴퓨터學 碩士學位 論文으로  
提出함

2021年 12月 30日

高麗大學校大學院

컴퓨터學科

朱 禮 珍 (印)



朱禮珍의 컴퓨터學 碩士學位論文 審査를  
完了함

2021年 12月 10日

委員長            이 희 조            (印)

---

委 員            오 학 주            (印)

---

委 員            허 준 범            (印)

---



# Abstract

With the increasing popularity of blockchain ecosystem and smart contracts, so do security concerns, in particular that of the propagation of vulnerable code, arise. Due to the immutable property of smart contracts and the high financial value a blockchain platform holds today, preventing the replication of vulnerable codes in smart contracts is an urgent problem. To accurately inquire the vulnerable smart contract codes propagated over the real-world blockchain ecosystem, we present PROSMART, a precise mechanism for scanning PROPagated vulnerable SMART contracts at large scale. PROSMART proposes a novel approach in detecting smart contract vulnerabilities at large scale, aiming to be the cornerstone of vulnerability detection in the fast-changing smart contract environment.



# Contents

Abstract

Contents

List of Figures

List of Tables

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                      | <b>1</b> |
| <b>2</b> | <b>Background and Motivation</b>         | <b>4</b> |
| 2.1      | Background and taxonomy . . . . .        | 4        |
| 2.1.1    | Ethereum and Smart contract . . . . .    | 4        |
| 2.1.2    | Solidity language . . . . .              | 4        |
| 2.1.3    | Code clone . . . . .                     | 5        |
| 2.2      | Motivation . . . . .                     | 5        |
| <b>3</b> | <b>Solidity language particularities</b> | <b>7</b> |
| 3.1      | Function visibility . . . . .            | 7        |
| 3.2      | Event functions . . . . .                | 8        |
| 3.3      | Address type conversions . . . . .       | 10       |
| 3.4      | Authorization functions . . . . .        | 10       |



|           |                                                |           |
|-----------|------------------------------------------------|-----------|
| <b>4</b>  | <b>Design</b>                                  | <b>11</b> |
| 4.1       | PROSMART detection method . . . . .            | 11        |
| <b>5</b>  | <b>Implementation</b>                          | <b>13</b> |
| 5.1       | Dataset . . . . .                              | 14        |
| 5.2       | Solidity language parser . . . . .             | 14        |
| 5.3       | Application of PROSMART . . . . .              | 16        |
| <b>6</b>  | <b>Evaluation</b>                              | <b>17</b> |
| 6.1       | Accuracy of PROSMART . . . . .                 | 17        |
| 6.2       | Performance . . . . .                          | 19        |
| <b>7</b>  | <b>Findings</b>                                | <b>22</b> |
| 7.1       | CVEs with incorrect information . . . . .      | 22        |
| 7.2       | Distributions of vulnerability types . . . . . | 24        |
| 7.3       | Vulnerability propagation timeline . . . . .   | 27        |
| 7.4       | Statistics on contract code clone . . . . .    | 29        |
| <b>8</b>  | <b>Limitations and Future Work</b>             | <b>30</b> |
| <b>9</b>  | <b>Related works</b>                           | <b>31</b> |
| <b>10</b> | <b>Conclusion</b>                              | <b>33</b> |
|           | <b>Bibliography</b>                            | <b>34</b> |
|           | 국문초록                                           |           |



# List of Figures

|     |                                                                                                                                                                                                                                               |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Examples “mintToken” functions with various syntaxes for representing Solidity particularities. All of them were extracted from real-world smart contracts. Highlighted parts show the changes made by each particularity.                    | 9  |
| 5.1 | PROSMART Implementation Schema . . . . .                                                                                                                                                                                                      | 13 |
| 7.1 | Example of a CVE report related to vulnerable contract codes . . . . .                                                                                                                                                                        | 23 |
| 7.2 | The most propagated vulnerable function timeline (1) date-wise and (2) volume-wise. Known vulnerable functions’ dates are marked in red. Numbers in between dates in (1) indicate the number of detected functions during the period. . . . . | 27 |
| 7.3 | The vulnerable “sweep” function of CVE-2018-14085. . . . .                                                                                                                                                                                    | 28 |





# List of Tables

|     |                                                                                                                                                    |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 6.1 | Precision of VUDDY and PROSMART. . . . .                                                                                                           | 19 |
| 6.2 | Performance of VeriSmart and PROSMART on the top 10 most replicated vulnerable contract codes. . . . .                                             | 20 |
| 6.3 | Performance of VeriSmart and PROSMART on the top 10 longest contract codes. . . . .                                                                | 21 |
| 7.1 | CVEs with incorrect information. . . . .                                                                                                           | 24 |
| 7.2 | Types of Vulnerable Functions. Self-named vulnerability types are asterisked. Rest of the vulnerability types are official CWE type names. . . . . | 25 |



# Chapter 1

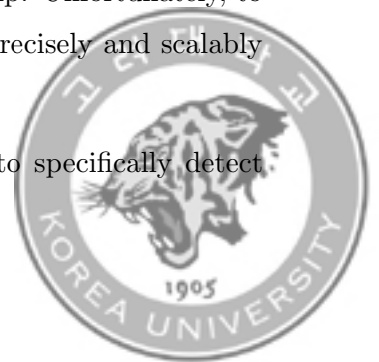
## Introduction

As blockchain based platforms gain a lot of interest, various possibilities for their application have emerged, *e.g.*, Ethereum [1]. In order to run applications, Ethereum requires a SMART CONTRACT, a programmed code for facilitating agreed obligations between untrusted parties, usually written in SOLIDITY LANGUAGE [2].

With the growing popularity of blockchain and the rapid development of smart contracts, on the other hand, several security concerns arise. One representative issue is the propagation of vulnerable code, which is naturally happened in the smart contract ecosystem where a piece of code is frequently reused in other contracts [3]. Such propagated vulnerabilities in a smart contract code can lead to a critical financial loss. Moreover, fixing the vulnerabilities of a smart contract once uploaded is a challenging problem owing to the *immutable* characteristic of smart contracts.

According to the existing study on smart contracts [3], 79.2% of the surveyed contracts were confirmed to be a clone of at least one another contract. In this context, in terms of code safety, accurately and rapidly determining whether propagated vulnerable code exists in a smart contract before it is deployed would be of great help. Unfortunately, to the best of our knowledge, existing approaches are not capable of precisely and scalably detecting vulnerable code clones in smart contracts.

**Limitation of existing approaches.** There exist tools aiming to specifically detect



vulnerabilities in smart contracts (*e.g.*, [4, 5, 6]). However, these tools are limited in terms of vulnerability detection coverage (*i.e.*, focusing only on arithmetic bugs), and requires performance improvement speed-wise for real-world applications. On the other hand, existing vulnerable code clone detection techniques (*e.g.*, [7, 8, 9]), can be used for detecting propagated vulnerable codes in smart contracts but they easily produce false negatives; this is because they hardly consider the unique code particularities appearing in the Solidity language (*e.g.*, they cannot handle the EVENT functions in the Solidity language, see Section 3.2).

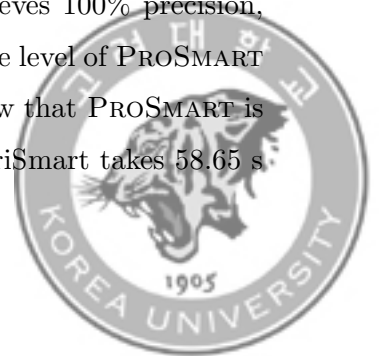
**Our approach.** To overcome such shortcomings, we propose PROSMART (PROPagated SMART contract vulnerable code clone scanning), which precisely and rapidly detects smart contract vulnerabilities better than any other existing vulnerability detection tools.

For accuracy, PROSMART pays sectionr attention to the Solidity language sectionri-  
ties in addition to the consideration of Type-1 and Type-2 code clones. Specifically, PROSMART focuses on four characteristics: 1) function visibility, 2) event functions, 3) address type conversions and 4) authorization functions (chapter 3). These particularities are absorbed into PROSMART’s abstraction method, explained in Section 4.1.

For scalability, PROSMART applies code clone detection mechanism in function-level, breaking down one function into tokens, then 1) labeling, 2) filtering, 3) abstracting and 4) hashing them.

**Evaluation.** To evaluate PROSMART, we collected a super large-scale real-world smart contracts dataset on Ethereum, exceeding 1.94 millions; this number is from 34.5 times to 83.6 times bigger than other works using real-world smart contract dataset [10, 3, 11]. In addition, known vulnerable functions have been collected from Common Vulnerabilities and Exposures (CVE) reports related to smart contract source codes. (details in Section 5.1)

The evaluation results (Section 6.1) show that PROSMART achieves 100% precision, while VUDDY shows 96.8% of precision. Meanwhile, the performance level of PROSMART has also been analyzed in comparison to VeriSmart. The results show that PROSMART is able to scan one whole contract code in 1.68 s on average, while VeriSmart takes 58.65 s



on average for the same dataset analysis.

**Findings.** Our large scale research on smart contract code vulnerabilities has led to some interesting facts worth noting. We explain how some information in CVE reports were incorrect, and how we have handled them (Section 7.1). We also describe the different vulnerability types of our collected vulnerable functions, and how the types must be handled code-clone-wise. Moreover, we display some interesting vulnerability propagation timelines and finally analyze the statistics on smart contract code clones.

**Contributions.** This paper makes the following three contributions:

1. We present PROSMART, a new vulnerable code clone detection method specifically for the Solidity language which provides scalability and better detection rate than any other existing tools.
2. We show that a lot of smart contract vulnerabilities propagate through code clones, and that existing vulnerable code clone detectors are incompatible with the Solidity language.
3. Our work’s experiments are based on a super large-scale real-world dataset: we have collected approximately 1.94 millions of contract source codes from Ethereum public network and carried out our experiments based on this dataset.



## Chapter 2

# Background and Motivation

In this chapter, we introduce background knowledge and then motivate PROSMART with examples.

## 2.1 Background and taxonomy

### 2.1.1 Ethereum and Smart contract

ETHEREUM [1] is a widely used public blockchain platform for various purposes such as trading valuable goods, global payments, and decentralized applications. To be able to carry out such actions, Ethereum requires a SMART CONTRACT, which is an open program uploaded on the blockchain and its execution is observed and confirmed by anyone. This contract takes the concrete form in programmed codes written in a specific programming language. There are several languages for writing smart contracts, but Solidity is by far the most widely used for Ethereum blockchain.

### 2.1.2 Solidity language

SOLIDITY [2] is an object-oriented programming language developed by Christian Reitwiessner and his team since late 2014, sponsored by the Ethereum Foundation [12]. As



the average 27 of days between consecutive releases [10] and the 0.x version numbers show, Solidity compiler is in a fast developing phase where many things are still experimented and modified. Throughout this paper, we refer to smart contract source code written in Solidity language as **contract code**; note that this does not refer to the byte code of a smart contract, but its *source code* written in a high-level programming language.

### 2.1.3 Code clone

CODE CLONES, also referred to as duplicate codes or similar codes, occur when a developer chooses to copy a part of existing codes instead of writing from scratch. Different ways exist to define and classify code clones, but this paper takes the following definition into consideration [13, 14, 15]:

- **Type-1: Exact clones.** Identical code fragments except for variations in whitespaces, layout and comments.
- **Type-2: Renamed clones.** Syntactical modifications added from Type-1 clones, showing further changes in identifiers, literals, and types.
- **Type-3: Restructured clones.** Structural modifications from Type-2 clones, showing changes such as addition, deletion, or rearrangements.
- **Type-4: Semantic clones.** Code fragments performing the same functionality but showing different syntactic implementation.

## 2.2 Motivation

To motivate our approach, we assert the usefulness of code clone detection for detecting smart contracts' vulnerabilities by showing that a small piece of vulnerable code has been propagated to numerous smart contracts.

In particular, we first collected Common Vulnerabilities and Exposures (CVE) [16] related to the smart contract; hundreds of reports related to the smart contract were



registered on CVE since 2018. After some manual filtering and confirmation works (details in Section 7.1), we have ended up with 493 functions related to the contract code vulnerabilities published before January 2021. Here, we confirmed that 362 (73.4%) of the 493 vulnerabilities made up clusters of Type-1 code clone, which were related to a vulnerable function named “`mintToken`”. Interestingly, this vulnerability has been reported continuously from 2017-03-11 to 2018-07-04 with different CVE IDs. Hence, such example of “`mintToken`” motivated us to further study propagated vulnerabilities in real-world smart contracts. Furthermore, from our experiment, PROSMART revealed that the vulnerable “`mintToken`” function still exists in 1,030 smart contracts that are currently serviced.

If the propagated vulnerable code can be precisely detected, it can proactively prevent constantly reported vulnerable code that has not been properly addressed for, as well as mitigates the propagation of vulnerable code to a number of real-world smart contracts, such as the case is for “`mintToken`”. Moreover, developers can ensure more secure contracts by rapidly reviewing whether their contract code contains known vulnerable code before deploying smart contracts.

Several existing tools (*e.g.*, [4, 5, 6]) hold the promise of analyzing contained vulnerabilities in the contract code but they do not focus on detecting propagated vulnerable code. In addition, they can only be applied to detect only a specific type of vulnerability (*e.g.*, arithmetic bugs), and also lack the performance to rapidly scan the smart contract. On the other hand, existing vulnerable code clone detection techniques (*e.g.*, [7, 8, 9]) that can, in principle, be applied to the detection of propagated vulnerable code in smart contracts, overlooks the particularities of the Solidity language, thereby yields many false negatives. Hence, there is a need for precise vulnerable code clone detection techniques specialized in smart contracts, to mitigate security threats caused by propagated vulnerable code widespread in real-world smart contracts.



## Chapter 3

# Solidity language particularities

During our study on Solidity code clone vulnerability, we discovered that there are some unique features of Solidity language that other existing vulnerability code clone detection (VCCD) tools are unable to catch. Here we introduce the particularities of Solidity language. The following particularities noted in this paper come from thorough study of Solidity vulnerable source codes (details in Section 7.1) along with the official changelog of Solidity [17] and Solidity v0.5.0 to v0.8.0 “Breaking Changes” from Solidity documentation [18].

### 3.1 Function visibility

In Solidity language, a function header follows the format shown in Listing 3.1 [19].

**Listing 3.1: Solidity grammar definition of function header.**

```
function functionName (parameterList)
(public, private, internal, external) or virtual or overrideSpecifier or returnsParameter
```

What we must note here is that the function visibility keywords (*i.e.*, public, private, internal, and external) can be placed anywhere after the parentheses of parameterLists. This means that the visibility can be defined right after the parameterLists, *or* after the





virtual keyword, *or* after the overrideSpecifier, *or* after the returnsParameter, *or* even not defined in which case the function will be public by default.

In previous tools which target programming languages such as C/C++ or Java, there was no need to consider such issue; this is because the position of the function visibility declaration is fixed for previous languages.

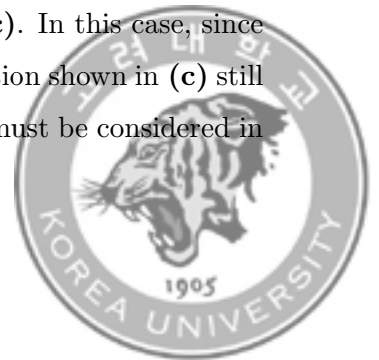
For example, Figure 3.1 (b) shows the “mintToken” vulnerable function with visibility keyword changes. In this case, the function indicated in (b) has the same meaning as the “mintToken” function in (a) regardless of whether there is a public keyword or not. Therefore, such a change where the semantic of function is maintained must be considered in the Solidity code clone detection.

## 3.2 Event functions

In Solidity language, there exists a type of function called EVENT functions; they are a sort of log or notification function which allows to print out some information that cannot be verified in the contract itself. Although it is a useful function, it has become one of the channels for TheDAO attack [20], where the fact that it is difficult to differentiate between event function calls and other function calls has been exploited.

To distinguish event function calls from the regular ones and mitigate this vulnerability, the Solidity team introduced a new keyword “**emit**” in compiler version v0.4.21 [21]. When using compiler of version v0.4.21 and after, it is recommended to include “emit” before an event function call. As a result, there are now two methods in calling an event function call: (1) call just like any other function calls and (2) call with “emit” prefixed to it.

Having a keyword specific for function calling is previously unseen in existing programming languages, and thus existing VCCD tools fail to classify the two methods as clones. An example changes in the Event function is shown in Figure 3.1 (c). In this case, since the “emit” keyword does not affect the vulnerable behavior, the function shown in (c) still has the same vulnerability as the vulnerable function in (a), which must be considered in



the Solidity vulnerable code clone detection.

---

```
1 (a) "mintToken" vulnerable function.
2 //Address: 0x070c9244a54353a0f9c43670b21856df2cc4e439
3
4 function mintToken(address target,
5     uint256 mintedAmount) onlyOwner {
6     balanceOf[target] += mintedAmount;
7     totalSupply += mintedAmount;
8     Transfer(0, this, mintedAmount);
9     Transfer(this, target, mintedAmount);
10 }
```

---

```
1 (b) Function visibility changes.
2 //Address: 0x22bb647119c45e52628e4cdebcb77da54152ecf3
3
4 function mintToken(address target,
5     uint256 mintedAmount) onlyOwner public {
6     balanceOf[target] += mintedAmount;
7     totalSupply += mintedAmount;
8     Transfer(0, this, mintedAmount);
9     Transfer(this, target, mintedAmount);
10 }
```

---

```
1 (c) Event function changes.
2 //Address: 0x6594cef9b8e3c10009beaaa25c599ef914fd11d0
3
4 function mintToken(address target,
5     uint256 mintedAmount) onlyOwner public {
6     balanceOf[target] += mintedAmount;
7     totalSupply += mintedAmount;
8     emit Transfer(0, this, mintedAmount);
9     emit Transfer(this, target, mintedAmount);
10 }
```

---

```
1 (d) Address type conversion.
2 //Address: 0xe8aefd62999994223272d5a06b083585a9da9c03
3
4 function mintToken(address target,
5     uint256 mintedAmount) onlyOwner public {
6     balanceOf[target] += mintedAmount;
7     totalSupply += mintedAmount;
8     emit Transfer(0, this, mintedAmount);
9     emit Transfer(address(this), target, mintedAmount);
10 }
```

---

Figure 3.1: Examples “mintToken” functions with various syntaxes for representing Solidity particularities. All of them were extracted from real-world smart contracts. Highlighted parts show the changes made by each particularity.



### 3.3 Address type conversions

Solidity language provides a particular “address” data type. This is unique to Solidity (and maybe other languages for implementing smart contracts) because smart contracts must handle **block addresses**.

These block addresses are of 20 byte size, and thus is also the size of address type. Such existence of a special “address” data type led to many explicit conversions to and from “address” value to and from “uint160”, integer literals, “bytes20” and contract typed values [22], to indicate that the input value is indeed an address.

### 3.4 Authorization functions

As Solidity written source codes build a smart *contract*, they naturally handle a lot of privilege configurations, such as defining the contract owner, limiting a function to be run only by authorized users, etc. Concretely, contract owner configuration is usually done in constructor functions, and function execution privilege is set by defining a modifier.

Constructors [23] in Solidity are functions executed only when the contract is created. They typically contain initializations necessary for the contract, including the selection of the contract owner. Prior to v0.4.22 of Solidity, constructors were defined as functions with the same name as the contract. Since v0.5.0, this syntax was deprecated and using “constructor” keyword for defining constructor functions became mandatory. Modifiers [24] are functions generally used to check for certain conditions before executing the function. They are applied to functions by including the modifier name at the function header.



# Chapter 4

## Design

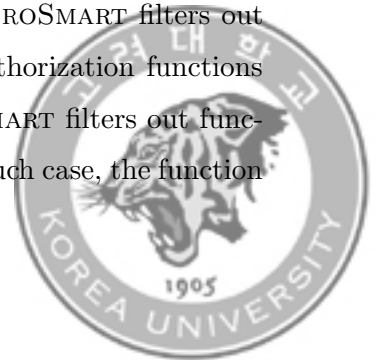
In this chapter, we describe the mechanism of PROSMART, a smart contract code clone detecting method focusing on scanning propagated vulnerabilities at large scale. Specifically, PROSMART is greatly inspired by VUDDY’s four levels of abstraction in function level [7], but has seen some changes to accommodate the particularities of Solidity language.

### 4.1 ProSmart detection method

We break down the mechanism into four steps and explain each in detail.

**Labeling.** PROSMART receives tokens of a function as input. These tokens must be labeled with specific token information necessary for four-level abstraction: function parameters, local variables, data types and function calls. On top of that, PROSMART requires additional labels related to the Solidity particularities: *function visibility*, *‘emit’ keyword*, *‘constructor’ keyword*, *type conversion*, *contract names* and *function name*.

**Filtering.** Before heading on to abstraction of a given function, PROSMART filters out constructor functions. This step is included to inspect whether authorization functions explained in Section 3.4 are properly protected. Concretely, PROSMART filters out functions whose function name is found in the list of contract names. In such case, the function



is a constructor function and thus is perfectly protected. Such functions do not proceed to abstraction step.

**Abstracting.** PROSMART proceeds to abstraction if the given function passes filtration. Similar to VUDDY, PROSMART abstracts function parameters, local variables, data types, and function calls. The difference from VUDDY here is that first, ‘emit’ keywords are removed to abstract event functions explained in Section 3.2. In addition, ‘public’ and ‘external’ visibility keywords are removed to treat visibility non-specified functions, ‘public’ specified functions, and ‘external’ specified functions together, because the three configured functions are callable by anyone. Also, this evades the function visibility position problem mentioned in Section 3.1 Last, data type conversions are abstracted. This step is crucial for Solidity source codes, due to the particularity explained in Section 3.3

**Hashing.** Finally, PROSMART generates a ‘fingerprint’ of the function through hashing just like VUDDY, but again, there are some differences. First difference from VUDDY is that PROSMART uses SHA256 hashing instead of MD5 hashing. This is simply because our dataset for comparison is extraordinarily large, which worried us for possible hash duplicates. If we assume a contract code has 20 functions (excluding constructor functions) on average, we have 38.8 millions of functions to compare. Besides, this change does not harm speed - the purpose of hashing - as proved in Section 6.2. Second, function headers are included in the preprocessed string for hashing. In VUDDY, function header was only used to extract function parameters then excluded for hashing. Such measure is taken to fully consider the Solidity particularity of Section 3.4, since the measure taken at Filtration step is not enough to ensure that all safe authorization functions have been filtered. Here by including the function header for hashing, PROSMART is able to differentiate authorization functions protected by a modifier, or functions prefixed by ‘constructor’ keyword.



## Chapter 5

# Implementation

In this chapter, we first introduce dataset construction. We then discuss how we built a Solidity parser, and further describe the implementation of PROSMART. The overview of PROSMART architecture is depicted in Figure 5.1.

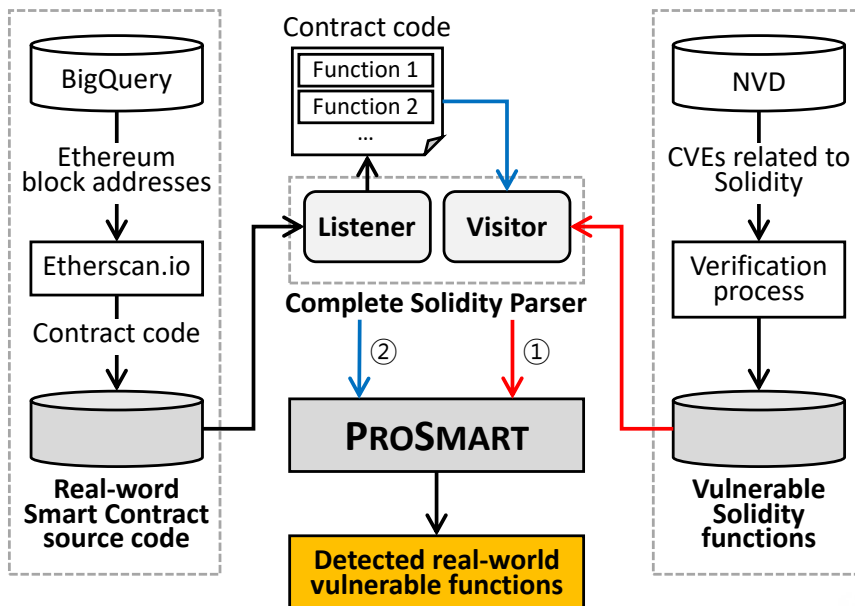


Figure 5.1: ProSmart Implementation Schema



## 5.1 Dataset

**Contract code collection.** As explained in Section 2.1.1, all blocks of Ethereum blockchain has a unique address assigned to them. So first, we collected all address values of Ethereum public net dating up to June 2020. This information was available on Google Cloud Platform’s BigQuery [25], collected by ethereum-etl [26]. As a result, approximately 118 millions of addresses were initially collected. Next, We used an API provided by Etherscan which returns verified contract source codes when a verified contract address is given [27]. We called this API for each of the 118 millions of addresses, and when the given address was verified, the API returned the corresponding verified source code information. In the end, we collected approximately **1.94 millions** of real-world smart contract source code. This process is illustrated on the left side of Figure 5.1.

**Vulnerable Solidity function collection.** We have collected Common Vulnerabilities and Exposures (CVE) reports related to smart contract source code vulnerability to use as our base vulnerable contract code dataset for analysis. To do so, we have searched for keywords such as “contract” and “ethereum” in the CVE database, then manually inspected the picked-up data. During this manual work, any vulnerabilities unrelated to the source code (e.g. CVE-2020-26265: Geth vulnerability) have been removed. Thus, we ended up with 520 CVE IDs related to smart contract source code, which gave 493 of vulnerable functions after careful inspection of each CVE ID, process for which is explained in details in Section 7.1.

## 5.2 Solidity language parser

To parse functions from the collected contract code, we needed a Solidity parser. We have investigated existing works related to the Solidity parser such as SmaCC [28], PASO [29] or SmartEmbed [11], and discovered that all of the parsers derived from one source: Solidity grammar for ANTLR4 built by Federico Bond [30]. Although the original repository is not getting developed anymore, the grammar .g4 source code has been



taken by the ConsenSys Diligence team [31], and then by the Solidity team [32], where it is currently maintained. Furthermore, there is a spin-off version of Federico Bond’s repository [33].

We have tried applying both the spin-off version and the Solidity team’s version on the collected contract code, but neither of them worked perfectly. Taking a closer look into the parsing errors, we found out the reason why: 99.2% of the collected contract code were compiled with version lower than v0.6.0, but neither of the two parsers were compatible with such old versions.

In the end, we built our own complete Solidity parser which would parse all Solidity compiler versions up to v0.8.0, released in December 2020.

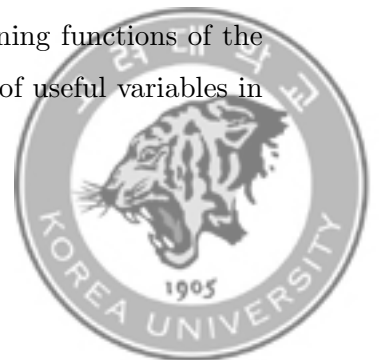
**Complete Solidity parser in ANTRL4.** The complete Solidity parser is based on the spin-off version. We changed the parser to accept all Solidity compiler versions. Most of the changes were based on the official changelog of Solidity [17] and Solidity v0.5.0 to v0.8.0 “Breaking Changes” from Solidity documentation [18].

We further changed the parser to ease the classifications necessary for code clone detection. For instance, we have grouped ‘public’, ‘private’, ‘external’, ‘internal’ keywords and labeled them as one `visibilityKeyword`, as shown in Listing 5.1.

**Listing 5.1: Definition for `visibilityKeyword` and its usage in the complete Solidity parser.**

```
modifierList
: ( visibilityKeyword | VirtualKeyword |
    stateMutability | modifierInvocation |
    overrideSpecifier )* ;
...
visibilityKeyword : PublicKeyword | PrivateKeyword | ExternalKeyword | InternalKeyword ;
```

**Execution of the complete Solidity parser.** The complete Solidity parser is used in two operations: (1) parsing a real-world contract code and returning functions of the contract and (2) tokenizing a given function and returning a group of useful variables in detecting vulnerability.





Specifically, the first operation was implemented with a custom listener for the ANTLR4-generated Solidity parser, while the second part was done with a visitor. The complete Solidity parser’s ANTLR4 .g4 grammar file along with the listener and visitor for functionizing and tokenizing is made available to public at <https://github.com/yjkellyjo/Solidity-complete-parser>.

### 5.3 Application of ProSmart

Here we explain how the PROSMART mechanism shown in Section 4.1 is applied with the dataset of Section 5.1 and the parser of Section 5.2. As illustrated in Figure 5.1, PROSMART receives two independent dataset from the complete Solidity parser’s visitor: 1) vulnerable Solidity functions tokens set and 2) contract code functions tokens set.

In real application, #1 dataset is treated before handling #2 dataset. Specifically, 493 vulnerable functions are processed through Labeling, Filtering, Abstracting and Hashing (details in Section 4.1), which results in 493 matching SHA256 hash values. These hash values of course see many duplicates among them, as the “sweep” case shows in Section 7.3. All corresponding hash value is registered along with the vulnerable Solidity function record for future reference.

Next, PROSMART treats #2 dataset, where one set of function tokens is handled one by one. For instance, if a contract code defines 15 functions (including constructors), then 15 set of tokens are generated and PROSMART handles these tokens one set by one set. Again just like the vulnerable dataset, each set passes through the four steps of PROSMART, and a corresponding SHA256 hash value is returned. This value is then searched in the registered vulnerable hash set, and if the search result is positive, we define this real-world function vulnerable.



## Chapter 6

# Evaluation

In this chapter, we evaluate the accuracy and the performance level of PROSMART by comparing them to state-of-the-art techniques: VUDDY (a scalable vulnerable code clone detection tool [7]) and VeriSmart (a smart contracts safety verification tool [4]). We evaluated PROSMART on a machine running Ubuntu 18.04.5 LTS, with a 2.20GHz Intel Xeon CPU E5-2630 v4 and 16GB RAM.

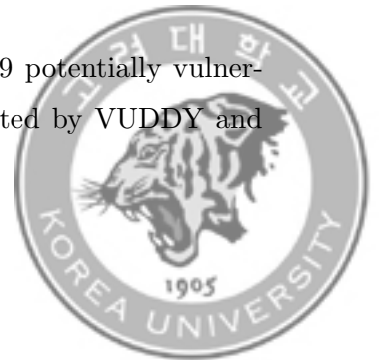
### 6.1 Accuracy of ProSmart

**Methodology.** We evaluate the accuracy of PROSMART by inspecting PROSMART and VUDDY’s vulnerable code clone detection results. As existing VUDDY implementation<sup>1</sup> only supports C/C++ languages, we have implemented VUDDY’s mechanism with the spin-off version of Solidity parser [33] for comparison. The experiment was conducted by running VUDDY and PROSMART implementation on the collected contract code, explained in Section 5.1. To evaluate PROSMART, we used the following three metrics: true positives (TP), false positives (FP), and precision ( $\frac{\#TP}{\#TP+\#FP}$ ).

**Accuracy measurement.** In the experiment, 404,753 and 392,229 potentially vulnerable functions (out of 1.94 million contract codes) have been detected by VUDDY and

---

<sup>1</sup><https://iotcube.net/hmark>



PROSMART respectively.

First, we assumed that the vulnerable code clones detected by both VUDDY and PROSMART are genuinely vulnerable (*i.e.*, TPs). Since vulnerable code clones detected by both VUDDY and PROSMART belong to Type-1 or Type-2 clones, we can decide that the semantics of the original vulnerable code were preserved in the detected clones; thus these cases can be treated as TPs. After this triage, only 13,326 vulnerable code clones detected by only VUDDY and 802 vulnerable code clones detected by only PROSMART remained. Next, since ‘Use of Cryptographically Weak Pseudo-Random Number Generator’ (PRNG) and ‘Integer Overflow’ (IO) vulnerability types especially fit well with detecting code clone as explained in Section 7.2, we marked those vulnerability typed functions as vulnerable without doubt. This left us with 13,042 and 28 functions respectively, all of them either ‘Improper Authentication’ (IA) or ‘Function Visibility’ (FV) vulnerability type. For both types, we determine that a function that satisfies any of the following conditions (*i.e.*, **C1** to **C4**) is safe.

**C1.** The function name corresponds with its contract name.

**C2.** The function is prefixed with ‘constructor’ keyword.

**C3.** The function visibility is ‘private’ or ‘internal’.

**C4.** The function is protected by a modifier which checks for the function caller’s approval status.

At last, in case a function did not meet any of the conditions, it was manually inspected and marked vulnerable if the detected function contained the vulnerability it was detected for. Else, it was marked safe.

Table 6.1 summarized the precision measurement results. We can see that all 28 IA or FV functions detected by PROSMART turned out to be vulnerable, thus TP. IA functions of PROSMART usually had a typo in their name or their contract’s name, which made the function callable by anyone instead of being a constructor function called only at the contract initialization. FV functions detected by PROSMART had ‘emit’ keyword in front



Table 6.1: Precision of VUDDY and ProSmart.

|                                            | VUDDY        | ProSmart      |
|--------------------------------------------|--------------|---------------|
| #Detected vulnerable code clones           | 404,753      | 392,229       |
| #Common detection                          | 391,427      |               |
| #Unique detection                          | 13,326       | 802           |
| #IA <sup>†</sup> and FV <sup>‡</sup> Types | 13,042       | 28            |
| #TP                                        | 391,712      | 392,229       |
| #FP                                        | 13,041       | 0             |
| <b>Precision</b>                           | <b>96.8%</b> | <b>100.0%</b> |

†: Improper Authentication, ‡: Function Visibility.

of the event function, making other detection methods blind. On the other hand, 13,041 out of 13,042 of IA or FV functions detected by VUDDY were actually safe, making them FP. Such high FP rate was due to VUDDY’s algorithm being unfit for Solidity; VUDDY discards function headers when generating hash, but the function headers in Solidity contain information such as the function visibility or the usage of modifiers. Consequently, VUDDY failed to check whether the detected function was in fact well protected, and produced high FP rate.

In the end, we confirm that PROSMART was able to show a 100% precision rate by carefully refining the detection mechanism for Solidity vulnerabilities.

## 6.2 Performance

**Methodology.** To evaluate performance, we examined PROSMART and VeriSmart’s speed. We used the existing implementation of VeriSmart<sup>2</sup> for comparison. Since VeriSmart considers each code line logic and determines vulnerability line by line, we have converted those numbers into function-level. The performance is experimented with two different datasets: (1) the top 10 most replicated real-world vulnerable contract codes, and (2) the top 10 longest real-world contract codes. Note that the top 10 most repli-

<sup>2</sup><https://iotcube.net/process/type/veris>



**Table 6.2: Performance of VeriSmart and ProSmart on the top 10 most replicated vulnerable contract codes.**

| ID           | #Addresses for<br>Vuln. Contracts | VeriSmart             |                        | ProSmart     |           |
|--------------|-----------------------------------|-----------------------|------------------------|--------------|-----------|
|              |                                   | Time <sup>†</sup> (s) | Vuln. <sup>‡</sup> (#) | Time(s)      | Vuln.(#)  |
| 1            | 372,218                           | 2.34                  | 0                      | 0.81         | 1         |
| 2            | 3,739                             | 10.82                 | 1                      | 0.65         | 3         |
| 3            | 176                               | 2.38                  | 0                      | 1.07         | 1         |
| 4            | 141                               | 1.44                  | 0                      | 1.07         | 1         |
| 5            | 133                               | 185.09                | 2                      | 0.65         | 1         |
| 6            | 115                               | 10.25                 | 1                      | 0.63         | 2         |
| 7            | 115                               | 201.63                | 7                      | 1.54         | 2         |
| 8            | 97                                | 2.62                  | 0                      | 4.06         | 2         |
| 9            | 91                                | 2.43                  | 0                      | 2.2          | 1         |
| 10           | 88                                | 184.46                | 2                      | 0.61         | 3         |
| <b>Total</b> | <b>376,913</b>                    | <b>603.46</b>         | <b>13</b>              | <b>13.29</b> | <b>17</b> |

†: Elapsed time, ‡: #Detected vulnerabilities.

cated vulnerable contract codes accounted for 19.4% of the contract codes collected (*i.e.*, 376,913 out of 1.94 millions). We used the second dataset for the purpose of evaluating the performance of each tool for long codes.

**Performance measurement.** Table 6.2 shows the experiment result of PROSMART and VeriSmart on the top 10 most replicated vulnerable contract codes. A number ID was given to each contract code in descending order of address counts to facilitate code reference.

What we must note here is the elapsed time. It is worth noting that PROSMART was 45 times faster than VeriSmart in terms of vulnerability detection performance. In detail, VeriSmart took a relatively long running time ranging from 1.44 to 201.63 s (averaged 60.35 s), while PROSMART showed substantially better performance with running time ranging from 0.61 to 4.06 s (averaged 1.33 s). We can notice that VeriSmart’s elapsed time for each contract code is roughly proportional to the number of vulnerabilities detected.



PROSMART, on the other hand, shows very short elapsed time regardless of the number of vulnerabilities.

**Table 6.3: Performance of VeriSmart and ProSmart on the top 10 longest contract codes.**

| Contract    | VeriSmart             | ProSmart     |
|-------------|-----------------------|--------------|
| Code Length | Time <sup>†</sup> (s) | Time(s)      |
| 243,547     | 184.24                | 3.75         |
| 242,731     | 2.31                  | 0.73         |
| 228,376     | 2.22                  | 0.36         |
| 223,737     | 2.17                  | 1.28         |
| 217,622     | 1.24                  | 2.06         |
| 180,616     | 1.57                  | 2.05         |
| 180,049     | 2.10                  | 2.37         |
| 178,077     | 2.36                  | 0.79         |
| 174,696     | 184.98                | 3.37         |
| 172,306     | 186.33                | 3.46         |
| 2,041,757   | 569.52                | <b>20.29</b> |

†: Elapsed time.

Table 6.3 shows the result of PROSMART and VeriSmart on the top 10 longest contract codes. The length of a contract code was determined by measuring the string length of a code where comments are removed and whitespace are normalized. It is worth noting that PROSMART’s total elapsed time for the 10 longest contract codes exceeded that of the 10 most replicated codes by only 7 s, while VeriSmart maintained its long total elapsed time. In the end, PROSMART was still 28 times faster than VeriSmart even for long contract codes, able to inspect about a hundred thousand code length in 1 s.

We can conclude that no matter how long the given code is, PROSMART scalably verifies the code in a very short time, which makes PROSMART convenient for real-world applications.



# Chapter 7

## Findings

In this chapter, we introduce some interesting facts worth noting, revealed in our experiments.

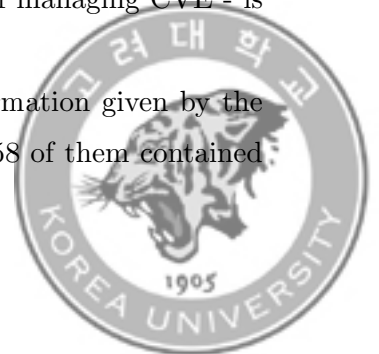
### 7.1 CVEs with incorrect information

In this section, we give detailed explanation on how the mentioned 520 CVE IDs in Section 5.1 were reduced to 493 of vulnerable functions. Before we go into detailed explanations, note two facts about CVE reports related to vulnerable contract codes.

- *Description* is mostly written in the following format: “The `function_name` function of a smart contract implementation for `contract_name`, and Ethereum token, has `vuln.type` that `detailed explanation`.”
- *References* usually records some urls that direct to a related contract address code information provided by Etherscan.

An example from the MITRE [34] - the organization in charge of managing CVE - is shown in Figure 7.1.

We aimed to extract vulnerable source code data from the information given by the 520 CVE IDs in Section 5.1. During this step, we discovered that 58 of them contained



| CVE-ID                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CVE-2018-13499</b>                                                                                                                                                                                                                                                                                                                                                                                                                                            | <a href="#">Learn more at National Vulnerability Database (NVD)</a><br>• CVSS Severity Rating • Fix Information • Vulnerable Software Versions • SCAP Mappings • CPE Information |
| Description                                                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                  |
| The mintToken function of a smart contract implementation for Crowdsale, an Ethereum token, has an integer overflow that allows the owner of the contract to set the balance of an arbitrary user to any value.                                                                                                                                                                                                                                                  |                                                                                                                                                                                  |
| References                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                  |
| <b>Note:</b> <a href="#">References</a> are provided for the convenience of the reader to help distinguish between vulnerabilities. The list is not intended to be complete.                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                  |
| <ul style="list-style-type: none"> <li>• <a href="https://github.com/BlockChainsSecurity/EtherTokens/blob/master/GEMCHAIN/mint%20integer%20overflow.md">MISC:https://github.com/BlockChainsSecurity/EtherTokens/blob/master/GEMCHAIN/mint%20integer%20overflow.md</a></li> <li>• <a href="https://github.com/BlockChainsSecurity/EtherTokens/tree/master/Crowdsale">MISC:https://github.com/BlockChainsSecurity/EtherTokens/tree/master/Crowdsale</a></li> </ul> |                                                                                                                                                                                  |

**Figure 7.1: Example of a CVE report related to vulnerable contract codes**

incorrect information. Details on such cases are classified in the Table 7.1 along with a representative example.

First, 42 out of 520 CVE IDs gave references to a contract code which actually did not contain the mentioned vulnerable function (*e.g.* CVE-2018-13173: “EliteShipperToken (ESHIP)”s “mintToken” function<sup>1</sup>). Two among them had the described vulnerable function, but commented out (*e.g.* CVE-2018-13526: “WangWangToken”s “mintToken” function<sup>2</sup>). Five pairs of two CVE IDs indicating one same vulnerable function of a same contract address has been found. This means 10 CVE IDs exists where only 5 would have been sufficient. For instance, CVE-2018-13225 and CVE-2018-13226’s descriptions seem to indicate two different contracts MyYLC and YLCToken, but their reference links actually direct to one same contract address<sup>3</sup>. In addition, three CVE IDs directed to contract codes containing the mentioned function names, but the functions were disclosed as *safe* after our inspection (*e.g.* CVE-2018-13680: “LexitToken”s “mintToken” function<sup>4</sup>). Last, CVE-2018-14576 mentioned “mintToken” function of “SunContract” as vulnerable, but the actual vulnerable function was found out to be “mintTokens”, misspelling the function name. We have reported this issue to MITRE, and it was corrected as of September 2021.

<sup>1</sup><https://etherscan.io/address/0x06bead2ead661b51307b646f7419d5284330c135#code>

<sup>2</sup><https://etherscan.io/address/0x47ced6e2c88355c75fedcbc1056bee39c25b4edf#code>

<sup>3</sup><https://etherscan.io/address/0x9f03d94bf3545153cb28014bc7d1e7177ca99034#code>

<sup>4</sup><https://etherscan.io/address/0xaa031595d2d9b82847a5df3390c6395839b273d0#code>





**Table 7.1: CVEs with incorrect information.**

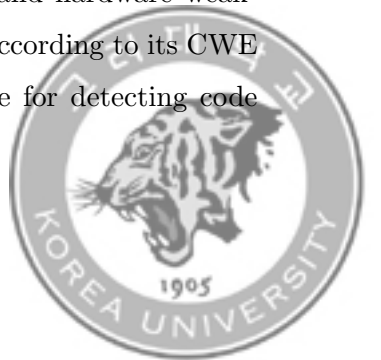
| Type of Error                          | Number of CVE IDs | Example CVE IDs                 |
|----------------------------------------|-------------------|---------------------------------|
| Function acutally not existing         | 42                | CVE-2018-13173                  |
| Commented out vulnerable function      | 2                 | CVE-2018-13526                  |
| Two CVE IDs for one same vulnerability | 10                | CVE-2018-13225 & CVE-2018-13226 |
| Actually not vulnerable function       | 3                 | CVE-2018-13680                  |
| Wrong function name                    | 1                 | CVE-2018-14576                  |
| <b>Total</b>                           | 58                | -                               |

Note that most of the mentioned CVEs with incorrect information were referenced with a github url<sup>5</sup> which reported hundreds of similar vulnerable functions usually named “mintToken” or “sell”. Many of the reported vulnerabilities contained correct information and are registered on CVE, but some of them, as described here, contained wrong information. In the end, we have collected 493 vulnerable functions. This dataset will be open to public at the publication time.

## 7.2 Distributions of vulnerability types

In Section 7.1, we mention how the *Description* for CVE reports provides `vuln._type` information. This vulnerability type information provided is usually related to Common Weakness Enumeration (CWE), which lists types of software and hardware weaknesses [35]. In this section, we observe the 493 vulnerable functions according to its CWE type, and explain why certain vulnerability types are more suitable for detecting code

<sup>5</sup><https://github.com/BlockChainsSecurity/EtherTokens>



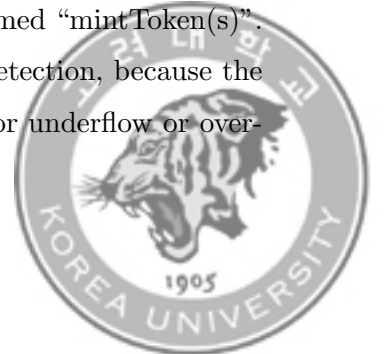
**Table 7.2: Types of Vulnerable Functions.** Self-named vulnerability types are asterisked. Rest of the vulnerability types are official CWE type names.

| Type of vulnerability                                               | Number of vulnerable functions | Type-1 clone detection efficiency |
|---------------------------------------------------------------------|--------------------------------|-----------------------------------|
| Integer Overflow (IO)                                               | 449                            | ○                                 |
| Incorrect Calculation (IC)                                          | 1                              | ○                                 |
| Replay Attack (RA)*                                                 | 12                             | ○                                 |
| Improper Input Validation (IIV)                                     | 7                              | ○                                 |
| Use of Insufficiently Random Values (IRV)                           | 1                              | ○                                 |
| Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG) | 11                             | ○                                 |
| Improper Authentication (IA)                                        | 6                              | △                                 |
| Function Visibility (FV)*                                           | 9                              | △                                 |
| <b>Total</b>                                                        | 496                            | -                                 |

clones than others. We determine a vulnerability type suitable for code clone detection when its detected Type-1 clones can be said vulnerable without doubt.

The number of vulnerable functions for each vulnerability type is summarized in Table 7.2. Note that the total does not sum up to 493, because some of the functions were charged for two CWE types (meaning two CVE IDs were also granted for one same function of one contract). There were also some functions not classified (CWE-noinfo) due to insufficient information. In such cases, we have named the vulnerability type by ourself from analyzing the case; such vulnerability types are asterisked in Table 7.2.

First notable fact is that there is a dominating number of “Integer Overflows” (IO) (449 of 496, 90.5%), and a majority (362) of the IO functions are named “mintToken(s)”. These vulnerable functions are typically efficient for Type-1 clone detection, because the vulnerability simply resides in an arithmetic operation unchecked for underflow or over-



flow. “Incorrect Calculation” (IC) type is IO type’s parent according to CWE Relationships [36]. Thus, similar reasoning for IO functions applies here and once a clone of IC typed function is detected, we can determine it as also vulnerable.

“Replay Attack” typed vulnerable functions of Solidity exploit precisely the fact that there exists duplicate codes in different contracts which check for the exact same conditions in the contract. This implies any Type-1 clone detected functions are also vulnerable. “Improper Input Validation” vulnerable functions typically have an issue in condition checks, which leads to unintended results. Such vulnerabilities are also efficient with Type-1 clones. “Use of Insufficiently Random Values” (IRV) vulnerability lies in insufficient randomness that allows attackers to determine the random number. The code for insufficient randomness would cause the same issue in other contracts, which explains that Type-1 clones of IRV typed functions are also vulnerable. Additionally, “Use of Cryptographically Weak Pseudo-Random Number Generator” (PRNG) is IRV type’s child type according to CWE Relationships [37], which indicates PRNG vulnerabilities are also suitable for Type-1 clone detection.

While the above vulnerability types can be easily determined as suitable, the rest cannot. “Improper Authentication” (IA) vulnerable functions in Solidity typically have the problem of *unprotected authorization*. More specifically, these functions grant some users certain administration power (*i.e.* change ownership of the contract), but are not properly protected, allowing attackers to gain unauthorized power over the contract. Such function clones’ vulnerability is not easy to verify, because for the function to be vulnerable, it must meet all following conditions:

- (1) The function must not be a constructor.
- (2) The function must be visible to public.
- (3) The function must not be protected by an authorization-checking modifier.

Last, “Function Visibility” (FV) functions have similar problems to IA functions; they compute some critical actions such as transferring tokens or modifying contract ownership, but are set as public or external, making them callable by anyone.



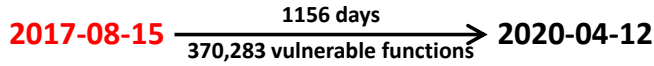
(1) Longest Timeline (date-wise)



PROSMART hash value:

1a7977e029c458be239d44b32f9b590d314f2f9397c234922b4a054810d430e1

(2) Largest Timeline (volume-wise)



PROSMART hash value:

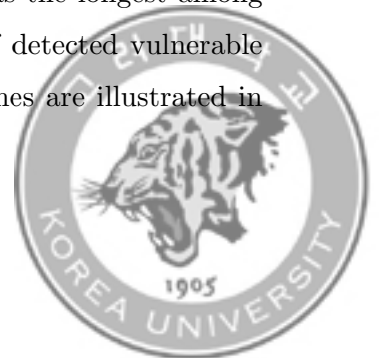
fb1a0a17d621a3ec3a5681f7dfcde40d23ce9990fc2f2cc0f78d0bcfd645016d

Figure 7.2: The most propagated vulnerable function timeline (1) date-wise and (2) volume-wise. Known vulnerable functions' dates are marked in red. Numbers in between dates in (1) indicate the number of detected functions during the period.

### 7.3 Vulnerability propagation timeline

In this section, we examine the timeline of vulnerable contracts. Specifically, vulnerable functions were grouped according to PROSMART Type-2 clusters, and lined up by function's verification date, which made one timeline for each Type-2 cluster. Verification dates have been collected by extracting the verified dates labeled by Ethereum when they verified a contract. Destructed verified contracts' source code was still available by Etherscan API, but their verification date was not unfortunately, so we could not collect verification dates for about half (53.7%) of the contract codes, because the corresponding verified contract were destructed. Thus, we made one timeline for each cluster with the available verification dates.

Here, we examine two timelines in detail: the most propagated vulnerable function's timeline (1) date-wise and (2) volume-wise. Most propagated date-wise signifies that the vulnerability of the cluster's propagation (end date - start date) was the longest among all clusters. Most propagated volume-wise indicates the number of detected vulnerable functions in the cluster was the largest among all. The two timelines are illustrated in Figure 7.2



---

```

1 function sweep(address _token, uint _amount)
2     returns (bool) {
3     (_amount);
4     return sweeperList.sweeperOf(_token)
5         .delegatecall(msg.data);
6 }

```

---

**Figure 7.3: The vulnerable “sweep” function of CVE-2018-14085.**

The longest timeline’s Type-2 cluster was that of hash<sup>6</sup>, which grouped 80 of “Improper Authentication” functions propagating for 1480 days, dating from 2016-03-24 to 2020-04-12. Similar to the PROSMART detected IA vulnerable functions case explained in Section 6.1, many of these functions had a typo in their name, misspelling a widely used contract name ‘Owned’ as ‘owned’. Moreover, only three among 80 were known vulnerable functions, deployed on 2018-05-17, 2018-05-22 and 2018-09-04. Among the three, the oldest disclosed vulnerability is CVE-2018-11329, published on 2018-05-22. Aggregating all information, we can conclude that hash<sup>7</sup> vulnerability stayed unknown for 789 day and was still propagated even after disclosure for 690 more days.

The largest timeline’s Type-2 cluster was that of hash<sup>8</sup>, which grouped 370,285 “Function Visibility” vulnerable functions propagating for 1156 days, dating from 2017-08-05 to 2020-10-04. An unbelievable truth here is that these functions were discovered from only one known vulnerable function ‘sweep’, of CVE ID: CVE-2018-14085. This vulnerable function is shown in Figure 7.3. Although this vulnerability must meet a certain condition in order to be manifested, it scores high in CVSS, indicating that once this condition is met, it may cause critical problems. This also means the newly detected 370,284 functions must be studied in detail to be determined actually vulnerable or not. However, mitigation for this vulnerability is as simple as just changing ‘sweep’ function’s visibility to private (or internal), or setting a modifier condition which checks for the caller’s authorization. This is because the vulnerability exploits the fact that ‘sweep’ function is public and unsupervised by modifiers, allowing the attacker to freely call the function.

---

<sup>7</sup>1a7977e029c458be239d44b32f9b590d314f2f9397c234922b4a054810d430e1

<sup>8</sup>fb1a0a17d621a3ec3a5681f7dfcde40d23ce9990fc2f2cc0f78d0bcfd645016d



## 7.4 Statistics on contract code clone

As shown in Section 6.1, 392,229 of real-world vulnerable functions have been precisely detected by PROSMART from only 493 of previously known vulnerable functions. As 392,229 includes the 493 known functions, we have newly discovered **391,736** vulnerable functions derived from 493 functions, the number approximately **795** times bigger than the known vulnerabilities.

Combining this result with the propagation timeline of Section 7.3, we conclude that the application of PROSMART mechanism can prevent such vast propagation over as long as 4.1 years with an extremely small dataset.



## Chapter 8

# Limitations and Future Work

PROSMART is designed to precisely and scalably scan real-world smart contract source codes for known vulnerabilities. As PROSMART mechanism is highly dependent on known vulnerability data collection, it may face difficulties when new type of smart contract vulnerability arises. We have yet to extend our dataset and mechanism by searching for various smart contract vulnerability sources other than CVE.

In addition, our study was limited to Solidity source codes in Ethereum public net, although Solidity is also used for smart contract implementation on other growing blockchain platforms such as Hyperledger Fabric [38]. In the future, we plan to extend our work's applications.

Finally, PROSMART is only a mechanism for now. We have yet to develop it into a real-world applicable program, where developers can make use of PROSMART instantly while programming in Solidity.

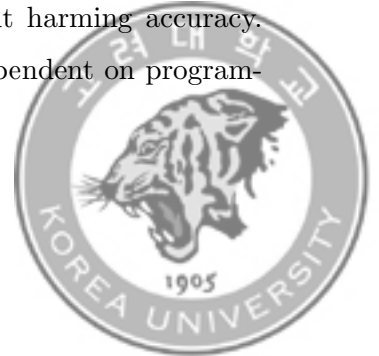


## Chapter 9

# Related works

**Vulnerability Detection in Smart Contracts.** There exist works seeking to specifically detect vulnerabilities in smart contracts by applying different methods in different scopes. VeriSmart [4] aims to precisely verify arithmetic safety of Ethereum smart contracts. VeriSmart achieves this by applying a new domain-specific algorithm specifically fit for arithmetic operations. As result, VeriSmart showed 0.41% of false positive rate in detecting known arithmetically vulnerable operations. However, the detection scope of VeriSmart is limited to only arithmetic operations, and the operation is typically slow due its preciseness. This static verification tool relates to PROSMART in that they seek to detect smart contract vulnerabilities, but differs from PROSMART in that its detection scope is restricted.

**Vulnerable Code Clone Detection.** Unlike static verification tools, clone detection techniques tend to treat a wider scope. VUDDY [7] is a representative vulnerable clone detecting approach of such tools. VUDDY aims to scalably detect Type-2 vulnerable code clones through function-level granularity and length-filtering technique. Such approach successfully reduced running time and increased scalability without harming accuracy. Despite such strengths, vulnerable clone detections are typically dependent on programming languages.





**Code Clone Detection in Smart Contracts.** There exist previous works on code clone detection of smart contracts. He *et al.* study code reuse practices in Ethereum smart contract ecosystem [39]. They examine smart contracts similarities, conduct a quantitative analysis of contract code reuses and note the spread of vulnerabilities due to contract code clones. Meanwhile, Kondo *et al.* conduct a case study of smart contract code clones in Ethereum platform [3]. They aim to quantify the amount of clones in Ethereum, understand key characteristics of clone clusters, and compare real-world smart contracts with open-source codes provided by OpenZeppelin. While both works display some statistics on vulnerable smart contract code clones, they are much more focused on analyzing the contract codes clones current state.



## Chapter 10

# Conclusion

In this work we introduced PROSMART, a novel precise approach of scanning smart contract source codes for vulnerabilities in large scale. We designed an accurate mechanism for Solidity source codes in particular, which is capable of precisely detecting propagated vulnerable codes of known vulnerable code fragments. To test our mechanism, we have collected every single verified contract source code on Ethereum public net deployed before June 2020, along with known vulnerable functions reported in CVE. Compared with state-of-the-art techniques in vulnerable code clone detection and smart contract safety verification, PROSMART showed better accuracy and performance level, particularly in precision and speed.



# Bibliography

- [1] G. Wood, “Ethereum: A secure decentralised generalised transaction ledger,” *Ethereum project yellow paper*, vol. 151, no. 2014, pp. 1–32, 2014.
- [2] Solidity Team, “Solidity programming language.” <https://soliditylang.org/>. (Accessed on 09/12/2021).
- [3] M. Kondo, G. A. Oliva, Z. M. Jiang, A. E. Hassan, and O. Mizuno, “Code cloning in smart contracts: a case study on verified contracts from the ethereum blockchain platform,” *Empirical Software Engineering*, vol. 25, no. 6, pp. 4617–4675, 2020.
- [4] IEEE, *VeriSmart: A highly precise safety verifier for Ethereum smart contracts*, 2020.
- [5] IEEE, *Verx: Safety verification of smart contracts*, 2020.
- [6] *SMARTTEST: Effectively Hunting Vulnerable Transaction Sequences in Smart Contracts through Language Model-Guided Symbolic Execution*, 2021.
- [7] IEEE, *Vuddy: A scalable approach for vulnerable code clone discovery*, 2017.
- [8] *MVP: Detecting Vulnerabilities using Patch-Enhanced Vulnerability Signatures*, 2020.
- [9] IEEE, *VGRAPH: A robust vulnerable code clone detection system using code property triplets*, 2020.
- [10] *Gap between theory and practice: An empirical study of security patches in solidity*, 2020.



- [11] Z. Gao, L. Jiang, X. Xia, D. Lo, and J. Grundy, “Checking smart contracts with structural code embedding,” *IEEE Transactions on Software Engineering*, 2020.
- [12] Solidity Team, “Solidity v0.1.0 turns 5! a walk down memory lane...” <https://solidity.ethereum.org/2020/07/08/solidity-turns-5/>. (Accessed on 07/12/2021).
- [13] C. K. Roy and J. R. Cordy, “A survey on software clone detection research,” *Queen’s School of Computing TR*, vol. 541, no. 115, pp. 64–68, 2007.
- [14] D. Rattan, R. Bhatia, and M. Singh, “Software clone detection: A systematic review,” *Information and Software Technology*, vol. 55, no. 7, pp. 1165–1199, 2013.
- [15] Q. U. Ain, W. H. Butt, M. W. Anwar, F. Azam, and B. Maqbool, “A systematic review on code clone detection,” *IEEE access*, vol. 7, pp. 86121–86144, 2019.
- [16] National Institute of Standards and Technology, “NVD - Vulnerabilities.” <https://nvd.nist.gov/vuln>. (Accessed on 07/13/2021).
- [17] Ethereum Team, “solidity/changelog.md at develop · ethereum/solidity.” <https://github.com/ethereum/solidity/blob/develop/Changelog.md>. (Accessed on 09/12/2021).
- [18] Ethereum, “Solidity v0.5.0 breaking changes — solidity documentation.” <https://docs.soliditylang.org/en/latest/050-breaking-changes.html>. (Accessed on 09/12/2021).
- [19] Ethereum, “Language grammar: function-definition — solidity 0.8.7 documentation.” <https://docs.soliditylang.org/en/v0.8.7/grammar.html?highlight=language%20grammar#a4.SolidityParser.functionDefinition>. (Accessed on 09/23/2021).
- [20] D. Siegel, “Understanding the dao attack.” <https://www.coindesk.com/markets/2016/06/25/understanding-the-dao-attack/>, June 2016. (Accessed on 09/05/2021).



- [21] Solidity Team, “Release version 0.4.21 · ethereum/solidity.” <https://github.com/ethereum/solidity/releases/tag/v0.4.21>. (Accessed on 07/05/2021).
- [22] Solidity Team, “Types: Address — solidity 0.8.7 documentation.” <https://docs.soliditylang.org/en/v0.8.7/types.html#address>. (Accessed on 09/22/2021).
- [23] Solidity Team, “Contracts: Constructors — solidity 0.8.7 documentation.” <https://docs.soliditylang.org/en/v0.8.7/contracts.html?highlight=constructor#constructors>. (Accessed on 09/22/2021).
- [24] Solidity Team, “Contracts: Function modifiers — solidity 0.8.7 documentation.” <https://docs.soliditylang.org/en/v0.8.7/contracts.html?highlight=modifier#function-modifiers>. (Accessed on 09/22/2021).
- [25] Google Cloud Platform, “Ethereum cryptocurrency – marketplace.” <https://console.cloud.google.com/marketplace/product/ethereum/crypto-ethereum-blockchain?pli=1&project=speech-261008>. (Accessed on 09/14/2021).
- [26] blockchain-etl, “ethereum-etl.” <https://github.com/blockchain-etl/ethereum-etl>. (Accessed on 09/14/2021).
- [27] Etherscan API Documentation, “Get contract source code for verified contract source codes.” <https://docs.etherscan.io/api-endpoints/contracts#get-contract-source-code-for-verified-contract-source-codes>. (Accessed on 09/10/2021).
- [28] *Solidity parsing using smacc: Challenges and irregularities*, 2017.
- [29] IEEE, *Paso: A web-based parser for solidity language analysis*, 2020.
- [30] F. Bond, “solidityj/solidity-antlr4: Solidity grammar for antlr4.” <https://github.com/solidityj/solidity-antlr4>. (Accessed on 09/12/2021).
- [31] ConsenSys Team, “ConsenSys/solidity-antlr4: Solidity grammar for antlr4.” <https://github.com/ConsenSys/solidity-antlr4>. (Accessed on 09/12/2021).



- [32] Ethereum Team, “solidity/docs/grammar at develop · ethereum/solidity.” <https://github.com/ethereum/solidity/tree/develop/docs/grammar>. (Accessed on 09/12/2021).
- [33] F. Victorio, “solidity-parser/antlr: Solidity grammar for antlr4.” <https://github.com/solidity-parser/antlr>. (Accessed on 09/12/2021).
- [34] The MITRE Corporation, “The MITRE Corporation.” <https://www.mitre.org/>. (Accessed on 09/18/2021).
- [35] The MITRE Corporation, “CWE - Common Weakness Enumeration.” <https://cwe.mitre.org/>. (Accessed on 09/21/2021).
- [36] The MITRE Corporation, “Cwe - cwe-682: Incorrect calculation (4.5).” <https://cwe.mitre.org/data/definitions/682.html>. (Accessed on 09/21/2021).
- [37] The MITRE Corporation, “CWE - CWE-330: Use of Insufficiently Random Values (4.5).” <https://cwe.mitre.org/data/definitions/330.html>. (Accessed on 09/21/2021).
- [38] *Hyperledger fabric: a distributed operating system for permissioned blockchains*, 2018.
- [39] Springer, *Characterizing code clones in the ethereum smart contract ecosystem*, 2020.



# 프로스마트: 스마트 컨트랙트(Smart Contract)의 취약점 전파에 대한 정밀한 대규모 스캐닝 메커니즘

이름: 주예진

학과: 컴퓨터학과

지도교수: 이희조

블록체인 생태계와 스마트 컨트랙트의 인기가 높아지면서 보안에 대한 우려, 특히 취약한 코드 전파에 대한 우려가 커지고 있다. 스마트 컨트랙트의 불변성과 블록체인 플랫폼이 오늘날 보유하고 있는 높은 재무 가치로 인해 스마트 컨트랙트에서 취약한 코드의 복제를 막는 것은 시급한 문제다. 실제 블록체인 생태계를 통해 전파되는 취약한 스마트 컨트랙트 코드를 정확하게 검출하기 위해 전파된 취약한 스마트 컨트랙트를 대규모로 스캔하기 위한 정밀한 메커니즘인 **프로스마트**를 제시한다. **프로스마트**는 빠르게 변화하는 스마트 컨트랙트 생태계에서 취약성 탐지의 초석이 되는 것을 목표로 스마트 컨트랙트 취약성을 대규모로 탐지하는 새로운 접근 방식을 제안한다.

